

Module-1 HTML5

Project Theme: Local Community Event Portal

A local city council wants a lightweight, browser-based portal to help residents register for events, check locations, and access basic services.

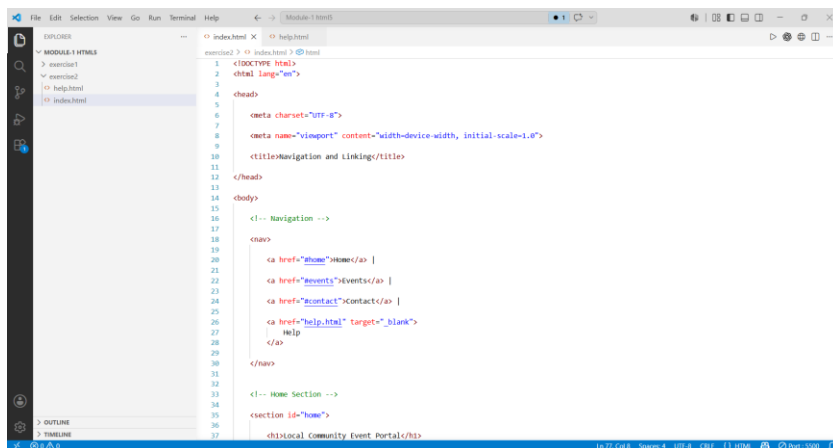
1. Create the HTML5 Base Template

Scenario: You're setting up the base document that every page on the portal will use.

Objective: Ensure semantic structure and compatibility across browsers.

Task:

- Use ,
- Add comments to label sections like "Navigation", "Main", "Footer"
- Save as index.html and open it in Chrome
- Inspect the document structure in Chrome Dev Tools



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6   <meta charset="utf-8">
7
8   <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10  <title>Navigation and linking</title>
11
12 </head>
13
14 <body>
15
16   <!-- Navigation -->
17
18   <nav>
19
20     <a href="#home">Home</a> |
21
22     <a href="#events">Events</a> |
23
24     <a href="#contact">Contact</a> |
25
26     <a href="help.html" target="_blank">
27       Help
28     </a>
29
30   </nav>
31
32   <!-- HOME Section -->
33
34   <section id="home">
35
36     <h1>Local Community Event Portal</h1>
```

Output:



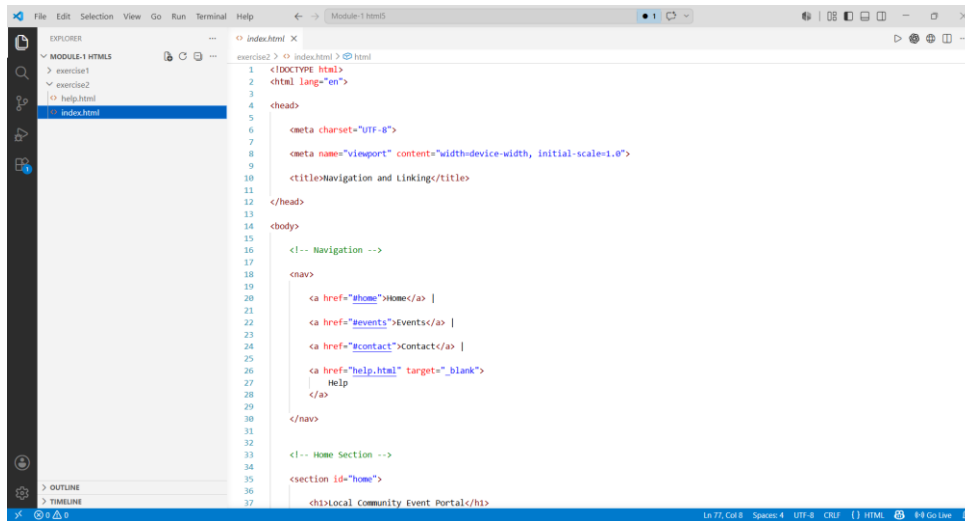
2. Navigation and Linking

Scenario: Users should navigate between "Home", "Events", and "Contact" sections.

Objective: Provide intuitive navigation and section-based references.

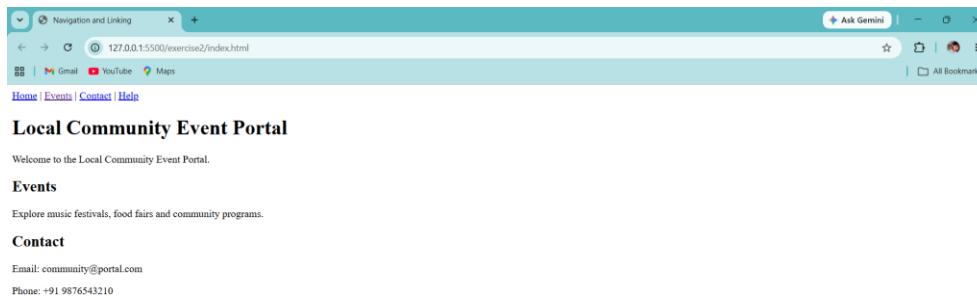
Task:

- Use with anchor tags [Events](#)
- Define matching IDs for each section like
- Add a link to an external help document using



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6   <meta charset="UTF-8">
7
8   <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10  <title>Navigation and Linking</title>
11
12 </head>
13
14 <body>
15
16   <!-- Navigation -->
17
18   <nav>
19
20     <a href="#home">Home</a> |
21
22     <a href="#events">Events</a> |
23
24     <a href="#contact">Contact</a> |
25
26     <a href="help.html" target="_blank">
27       | Help
28     </a>
29
30   </nav>
31
32
33   <!-- Home Section -->
34
35   <section id="home">
36
37     <h1>Local Community Event Portal</h1>
```

Output:



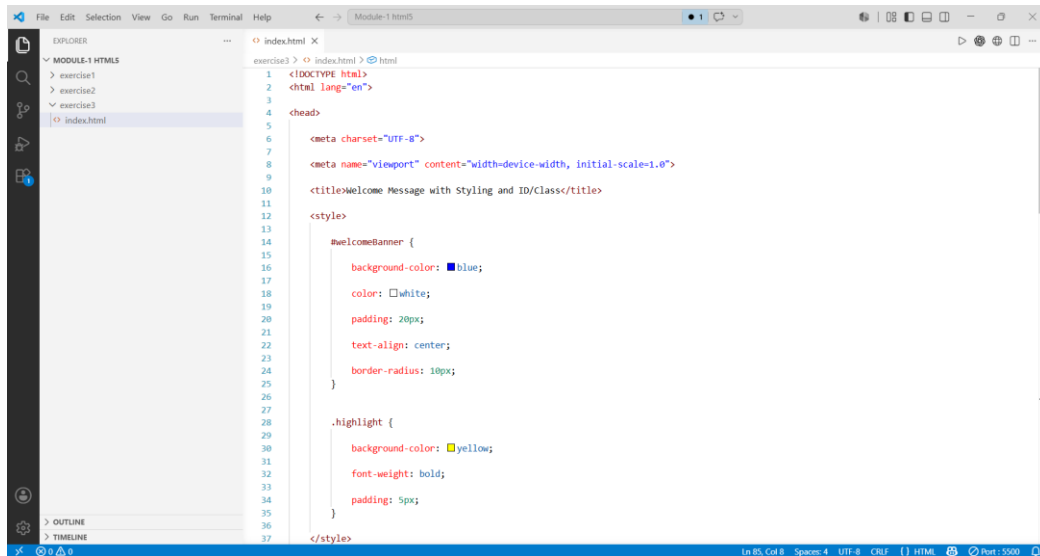
3. Welcome Message with Styling and ID/Class

Scenario: Display a welcome banner styled uniquely for a logged-in user.

Objective: Practice block/inline tags and differentiate id and class

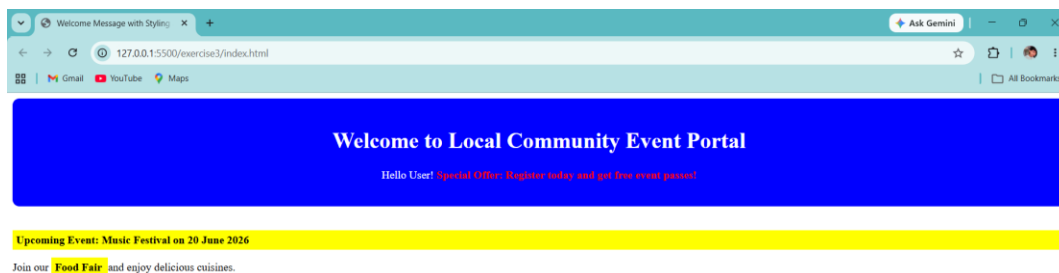
Task:

- Use and apply a blue background via internal CSS
- Use inline styles for a special offer (e.g., color red, bold)
- Apply the .highlight class to certain elements for visual emphasis



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Welcome Message with Styling and ID/Class</title>
11
12 <style>
13
14     #welcomeBanner {
15
16         background-color: blue;
17
18         color: white;
19
20         padding: 20px;
21
22         text-align: center;
23
24         border-radius: 10px;
25
26     }
27
28     .highlight {
29
30         background-color: yellow;
31
32         font-weight: bold;
33
34         padding: 5px;
35
36     }
37 </style>
```

Output:



4. Image Gallery for Community Events

Scenario: Show images from past events in a table layout.

Objective: Work with `<img>`., tables, and formatting tags.

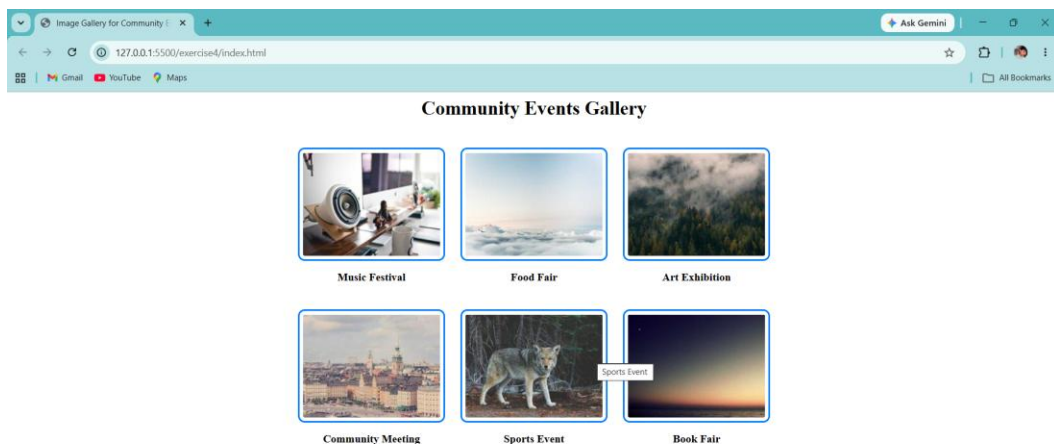
Task:

Use a `<table>` with 2 rows and 3 columns of `<img>`.tags

- Include alt, title, and style each image with borders using a class
- Add a caption to describe each event

```
File Edit Selection View Go Run Terminal Help
Module:1.html5
EXPLORER
  MODULE:1 HTMLS
    > exercise1
    > exercise2
    > exercise3
    > exercise4
      index.html
  OUTLINE
  TIMELINE
index.html X
exercise4 > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Image Gallery for Community Events</title>
11
12 <style>
13
14   table {
15     margin: auto;
16     border-spacing: 20px;
17   }
18
19   caption {
20     font-size: 28px;
21     font-weight: bold;
22     margin-bottom: 20px;
23   }
24
25   td {
26     text-align: center;
27   }
28
29   .eventImage {
30     width: 200px;
31   }
32
33
34
35
36
37
```

Output:



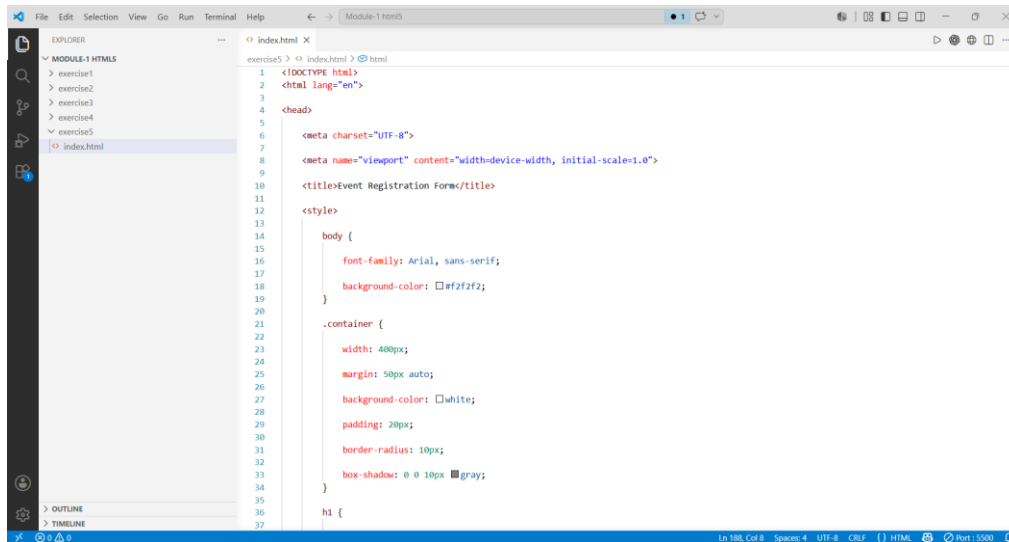
5. Event Registration Form

Scenario: Residents need to register for events.

Objective: Practice input types, validation, placeholder, autofocus, and output

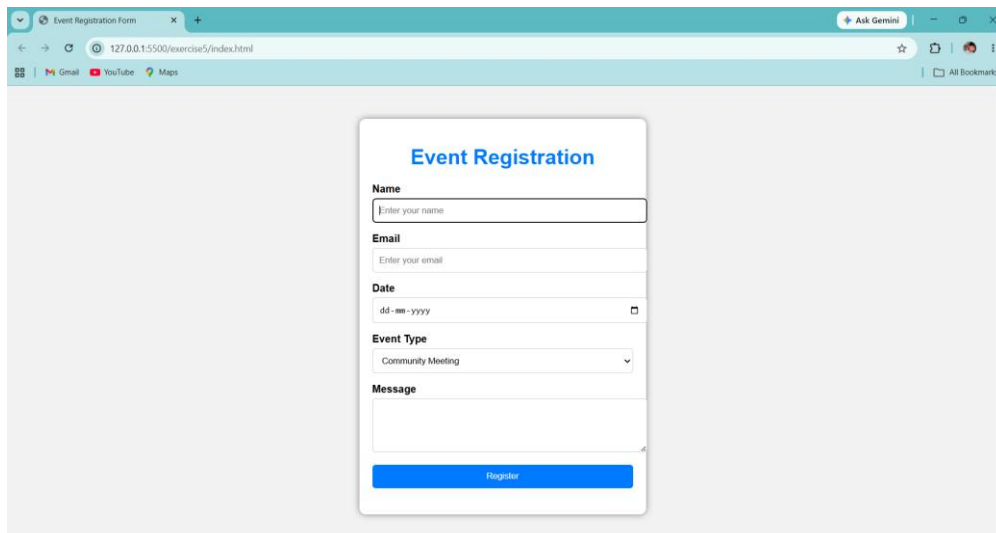
Task:

- Include fields: name (text), email (email), date (date), event type (select), message (textarea)
- Add placeholder, required, and autofocus
- Display a confirmation message using when the form is submitted
- Style the form using CSS



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Event Registration Form</title>
11
12 <style>
13
14   body {
15
16     font-family: Arial, sans-serif;
17
18     background-color: #f2f2f2;
19
20   }
21
22   .container {
23
24     width: 400px;
25
26     margin: 50px auto;
27
28     background-color: white;
29
30     padding: 20px;
31
32     border-radius: 10px;
33
34     box-shadow: 0 0 10px #gray;
35
36   }
37
38   h1 {
```

Output:



The screenshot shows a web browser displaying the Event Registration Form. The form is centered on a light gray background and has a white background with a blue border and a blue shadow. The form title is "Event Registration" in blue. The form fields are: Name (text input with placeholder "Enter your name"), Email (text input with placeholder "Enter your email"), Date (date input with placeholder "dd-mm-yyyy"), Event Type (select dropdown with "Community Meeting" selected), and Message (text area). A blue "Register" button is at the bottom of the form.

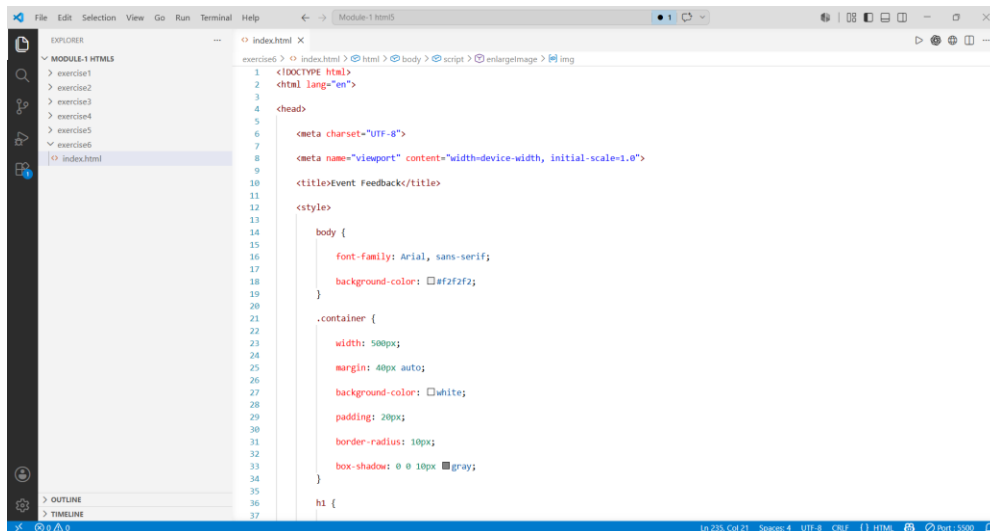
6. Event Feedback with Events Handling

Scenario: Collect real-time feedback and interactions from the user.

Objective: Handle blur, change, click, double-click, and keyboard events.

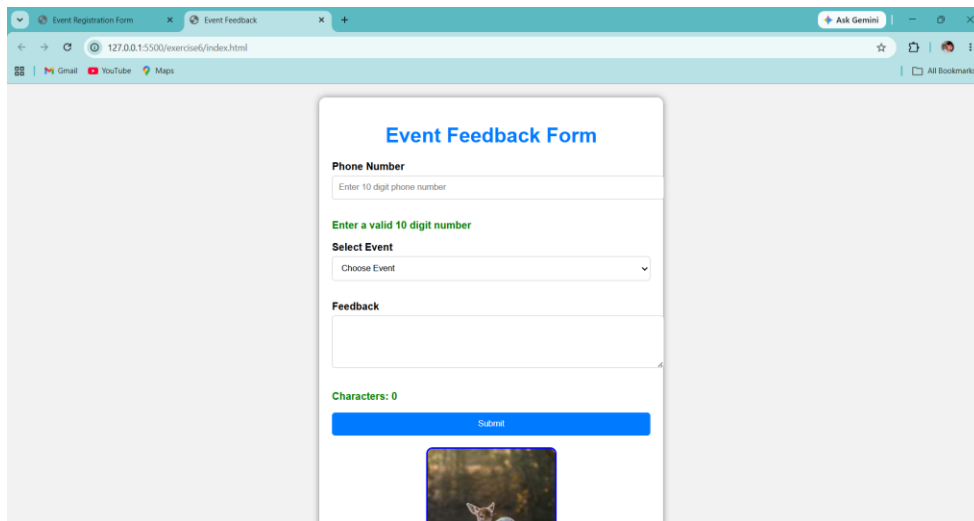
Task:

- Use onblur to validate a phone number field
- Use onchange on a dropdown to display the selected event fee
- onclick on a submit button to show a confirmation
- ondblclick on an image to enlarge it
- Capture key events in the feedback textarea and count characters



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Event Feedback</title>
11
12 <style>
13
14     body {
15
16         font-family: Arial, sans-serif;
17         background-color: #f2f2f2;
18     }
19
20     .container {
21
22         width: 500px;
23         margin: 40px auto;
24         background-color: white;
25         padding: 20px;
26         border-radius: 10px;
27         box-shadow: 0 0 10px gray;
28     }
29
30     h1 {
31
32
```

Output:



The screenshot shows a web browser displaying the "Event Feedback Form". The form is centered on a light gray background. It has a title "Event Feedback Form" in blue. Below the title, there are three main sections: "Phone Number" with a text input field and a green validation message "Enter a valid 10 digit number"; "Select Event" with a dropdown menu showing "Choose Event"; and "Feedback" with a large text area. Below the feedback text area, there is a green label "Characters: 0" and a blue "Submit" button. At the bottom of the form, there is a small image of a dog.

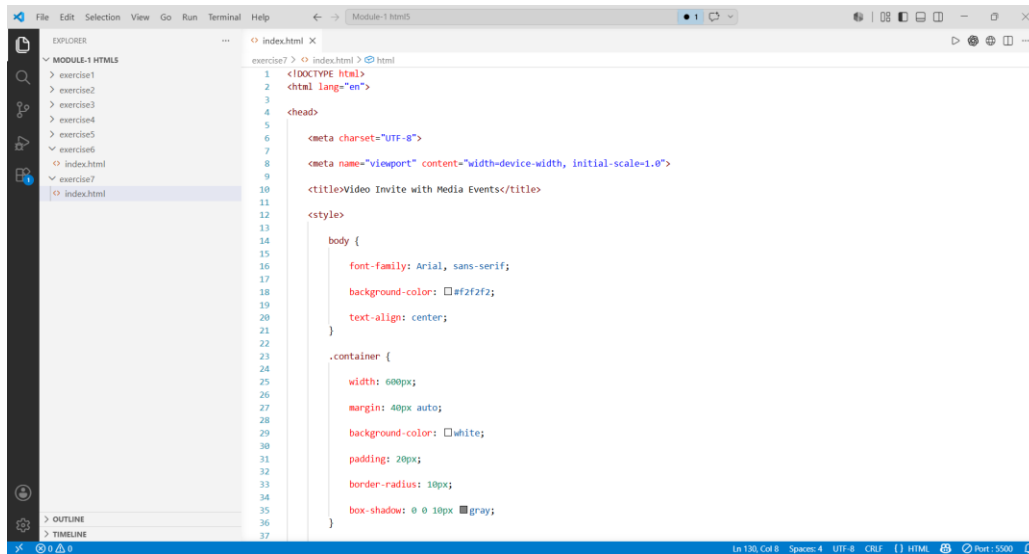
7. Video Invite with Media Events

Scenario: Show a short event promo video.

Objective: Work with and oncanplay event

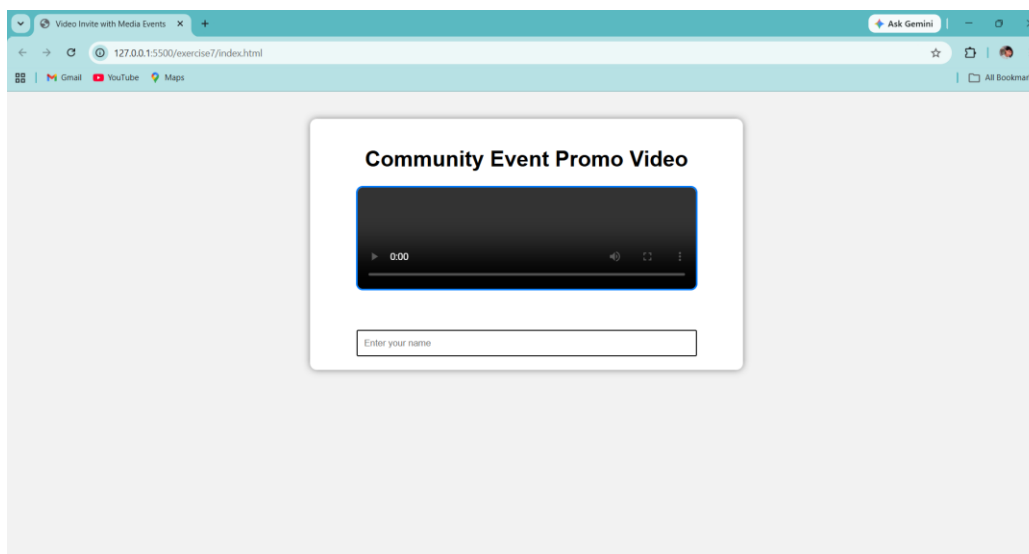
Task:

- Insert a element with source and controls
- Use oncanplay to display a message like "Video ready to play"
- Use onbeforeunload to warn users if they try to leave the form page unfinished



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Video Invite with Media Events</title>
11
12 <style>
13
14 body {
15
16     font-family: Arial, sans-serif;
17
18     background-color: #f2f2f2;
19
20     text-align: center;
21
22 }
23
24 .container {
25
26     width: 600px;
27
28     margin: 40px auto;
29
30     background-color: white;
31
32     padding: 20px;
33
34     border-radius: 10px;
35
36     box-shadow: 0 0 10px gray;
37
38 }
```

Output:



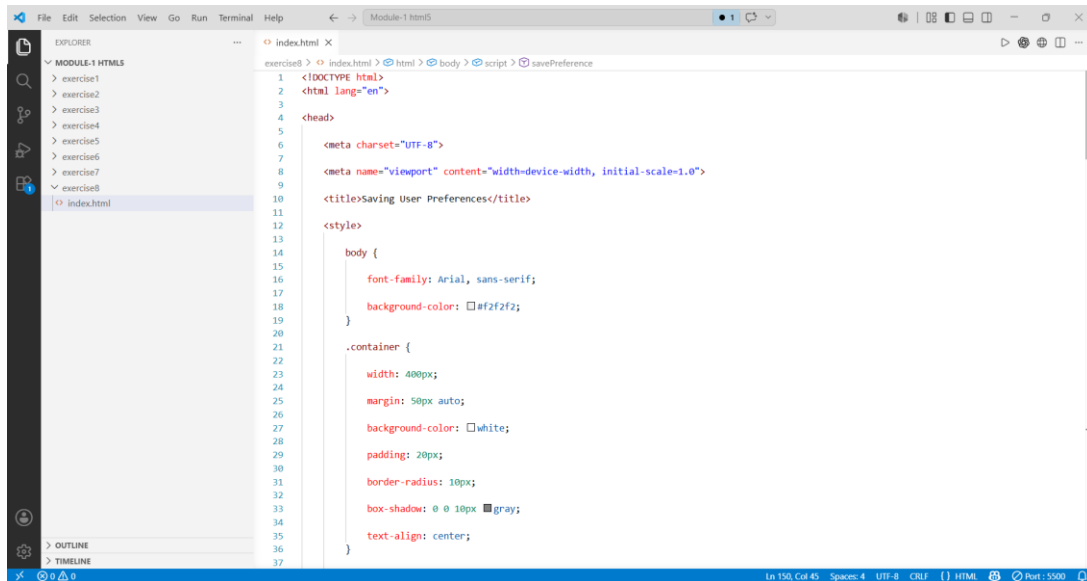
8. Saving User Preferences

Scenario: Store preferred event type for returning users.

Objective: Work with localStorage, sessionStorage, and deletion

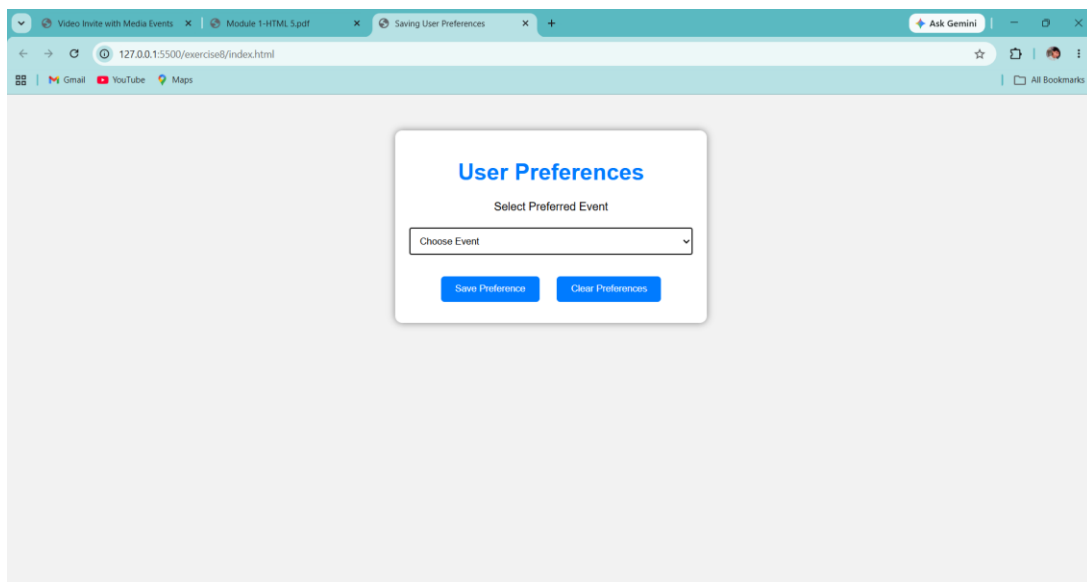
Task:

- Save selected event type in localStorage
- On reload, retrieve and pre-select it
- Add a "Clear Preferences" button that clears both localStorage and sessionStorage



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Saving User Preferences</title>
11
12 <style>
13
14 body {
15
16     font-family: Arial, sans-serif;
17
18     background-color: #f2f2f2;
19
20 }
21
22 .container {
23
24     width: 400px;
25
26     margin: 50px auto;
27
28     background-color: white;
29
30     padding: 20px;
31
32     border-radius: 10px;
33
34     box-shadow: 0 0 10px gray;
35
36     text-align: center;
37 }
```

Output:



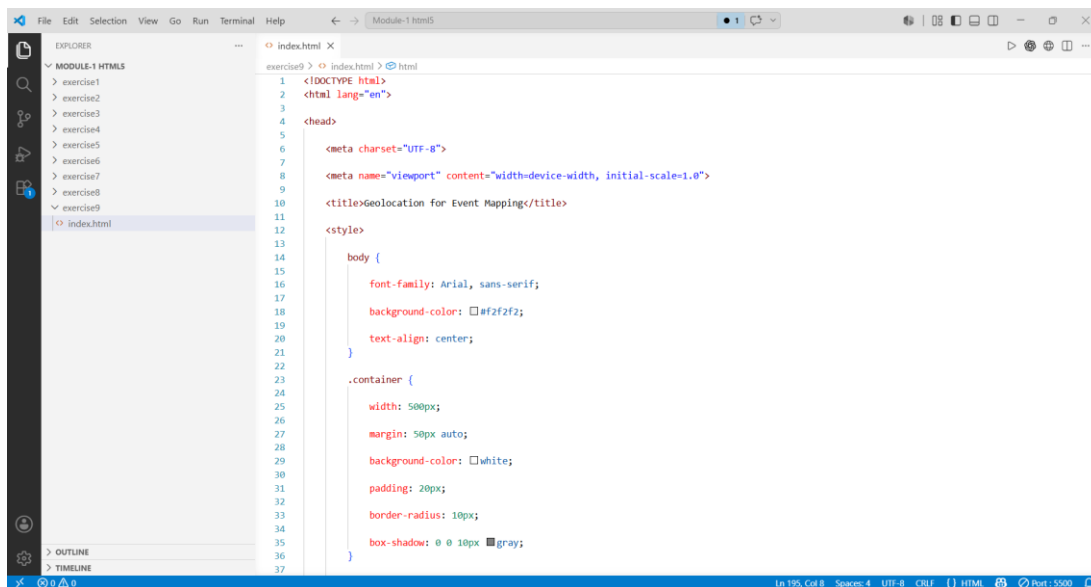
9. Geolocation for Event Mapping

Scenario: Locate the nearest event to the user.

Objective: Practice `geolocation.getCurrentPosition`, error handling, and options

Task:

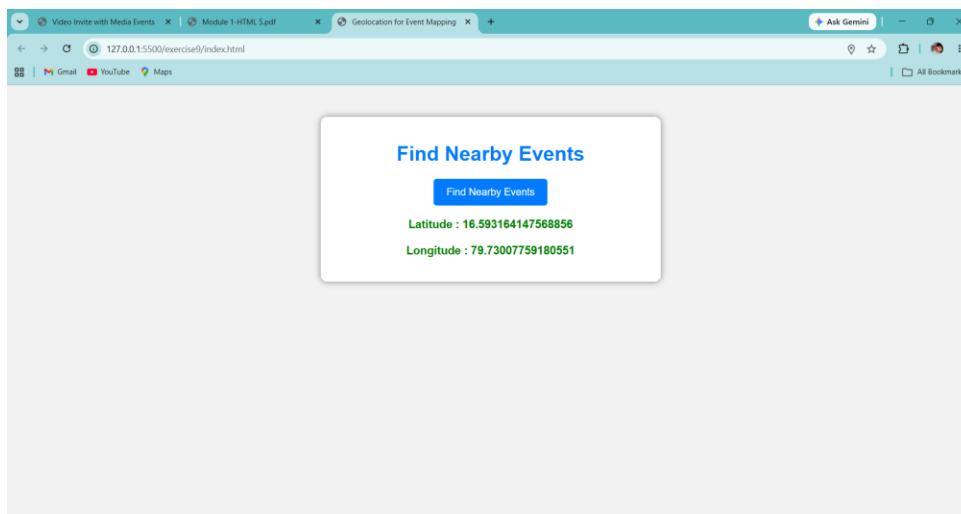
- Create a button “Find Nearby Events”
- On click, use `getCurrentPosition` to get and display coordinates
- Handle permission denial and timeouts
- Use high accuracy options



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6   <meta charset="UTF-8">
7
8   <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10  <title>Geolocation for Event Mapping</title>
11
12  <style>
13
14    body {
15
16      font-family: Arial, sans-serif;
17
18      background-color: #f2f2f2;
19
20      text-align: center;
21
22    }
23
24    .container {
25
26      width: 500px;
27
28      margin: 50px auto;
29
30      background-color: white;
31
32      padding: 20px;
33
34      border-radius: 10px;
35
36      box-shadow: 0 0 10px gray;
37    }

```

Output:



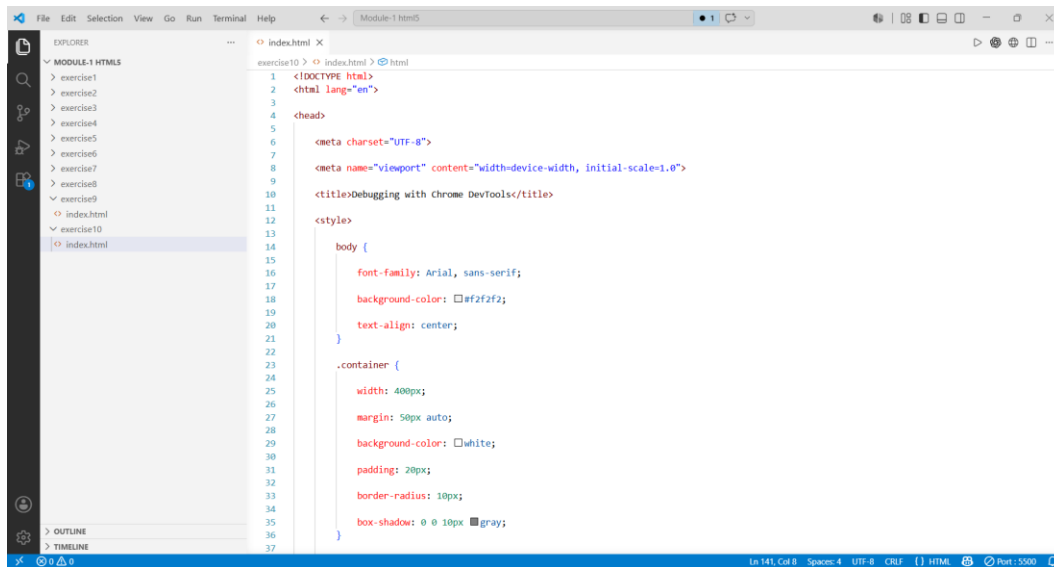
10. Debugging with Chrome DevTools

Scenario: A few users report layout issues and script errors.

Objective: Use Chrome DevTools and VS Code features to debug.

Task:

- Use “Inspect Element” to modify styles and experiment live
- Use the Console tab to view logs from your



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9
10 <title>Debugging with Chrome DevTools</title>
11
12 <style>
13
14   body {
15
16     font-family: Arial, sans-serif;
17
18     background-color: #f2f2f2;
19
20     text-align: center;
21   }
22
23   .container {
24
25     width: 400px;
26
27     margin: 50px auto;
28
29     background-color: #fff;
30
31     padding: 20px;
32
33     border-radius: 10px;
34
35     box-shadow: 0 0 10px #ccc;
36   }
37
```

Output:

